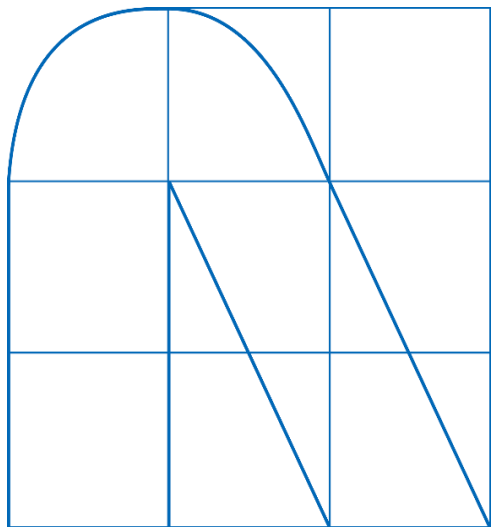


Info type: Confidential
Company: NTT Data Payment Services India Pvt Ltd
Info. owner: Product (PMG)



Refund API

Version 1.04

CONFIDENTIALITY DISCLAIMER

The information included in this document is confidential information relating to the business of NTT Data Payment Services, India(NDPS). It is being presented to you based on the understanding that it will not be used for any reason other than consideration of a commercial relationship with NDPS and, will not be used in connection with any decision to trade in securities of NDPS. Please be advised that any disclosure of the information contained in this document/presentation to any other person, or any use of this information in connection with the trading of NDPS securities, may be a violation.

This document, or any part of it, may not be reproduced, copied, circulated and/or distributed nor quoted without prior written approval from NDPS.

A. Document Information

Document Attributes	Information
Document Name	Refund API
Document Version	1.04
Owner	PMG
Author	Mukul Vishwakarma
Approved	

B. Revision Chart

This chart contains a history of this document's revisions.

Version	Primary Author	Description of Version	Date Completed	Reviewed By
1.0	Aviral Tripathi	Refund API	27/07/2022	Pavan N.
1.01	Aviral Tripathi	Signature generation string updated	16/06/2023	
1.02	Aviral Tripathi	Production URL updated	01/09/2023	
1.03	Aviral Tripathi	Updated Encryption Type	14/02/2025	Ashish T.
1.04	Mukul Vishwakarma	Introduction to API Secret Key and Modification in Endpoints	07/08/2026	

Contents

CONFIDENTIALITY DISCLAIMER	2
A. Document Information.....	2
B. Revision Chart.....	2
1. DESCRIPTION	5
Create API Secret Key.....	5
Data Validation	6
Create a Bearer Token	6
cURL	7
Python	7
Java.....	8
Node.js	9
PHP	10
Error Handling	12
Error Response Format.....	12
2. SINGLE PRODUCT REFUND	12
a) SINGLE PRODUCT SAMPLE REFUND REQUEST (OPEN DATA):.....	12
b) Specifications of API Request:.....	13
c) SINGLE PRODUCT SAMPLE REQUEST ENCRYPTED DATA:.....	14
a) SINGLE PRODUCT SAMPLE REFUND REQUEST (DECRYPTED):.....	14
b) SINGLE PRODUCT SAMPLE REFUND RESPONSE DATA (ENCRYPTED):	15
c) SINGLE PRODUCT SAMPLE RESPONSE DECRYPTED (OPEN DATA):	15
d) Specifications of API Response:	16
3. SINGLE PRODUCT PARTIAL REFUND.....	16
a) SINGLE PRODUCT SAMPLE PARTIAL REFUND REQUEST (OPEN DATA):.....	16
b) SINGLE PRODUCT SAMPLE PARTIAL REFUND REQUEST DATA (ENCRYPTED):-.....	17
c) SINGLE PRODUCT SAMPLE PARTIAL REFUND RESPONSE (ENCRYPTED):-	17
d) SINGLE PRODUCT SAMPLE PARTIAL REFUND RESPONSE (DECRYPTED):-.....	18
4. MULTI PRODUCT REFUND	18
a) MULTI PRODUCT SAMPLE REFUND REQUEST (OPEN DATA):-	18
b) MULTI PRODUCT SAMPLE REFUND REQUEST DATA (ENCRYPTED):-.....	19

c)	Specifications of API Request:.....	20
d)	MULTI PRODUCT SAMPLE REFUND RESPONSE DATA (ENCRYPTED):	21
e)	MULTI PRODUCT SAMPLE DECRYPTED REFUND RESPONSE (OPEN DATA):	21
f)	Specifications of API Response:	22
5.	MULTI PRODUCT PARTIAL REFUND.....	23
a)	MULTI PRODUCT SAMPLE PARTIAL REFUND REQUEST DATA (Open Data):-	23
b)	MULTI PRODUCT SAMPLE PARTIAL REFUND REQUEST DATA (ENCRYPTED):-	24
c)	MULTI PRODUCT SAMPLE PARTIAL REFUND RESPONSE DATA (ENCRYPTED):-.....	24
d)	MULTI PRODUCT SAMPLE DECRYPTED PARTIAL REFUND RESPONSE (DECRYPTED):-.....	24
6.	AES ENCRYPTION LOGIC	25
a)	AES Encryption Java Code:	25
7.	SIGNATURE GENERATION LOGIC	27
a)	Sample Request	28
b)	Generated Signature	28
c)	Hash Keys.....	28
d)	Encryption Key	28
e)	Signature Generation Java Code:	28
8.	UAT ENVIRONMENT DETAILS.....	29
9.	RESPONSE CODES	30

1. DESCRIPTION

The **Refund API** is provided to the merchant to raise refunds against successful transactions.

Request should be initiated, and the response is captured using the HTTP **POST** method.

a) URL: (Encrypted URL)

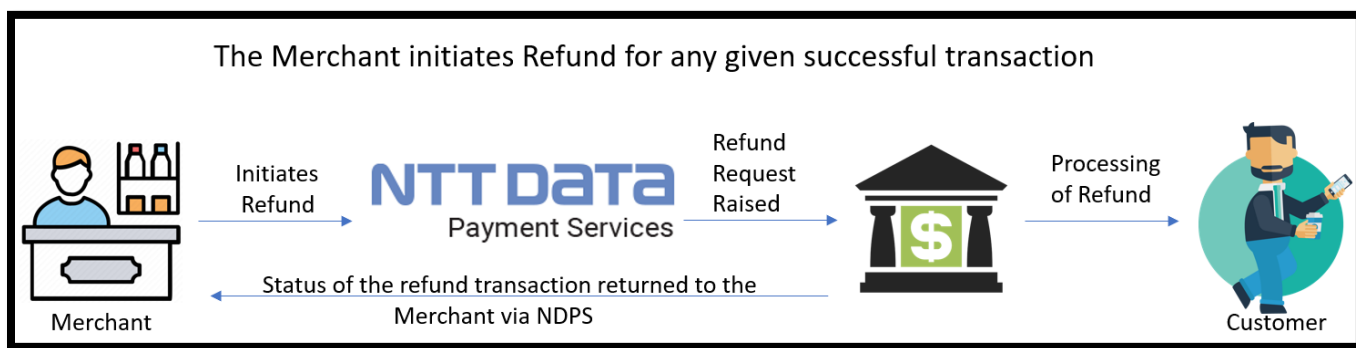
Production: <https://payment.atomtech.in/ots/payment/refund?merchId=7&encData=3efsgdfdfhgdhgggfbg>

UAT: <https://paynetzuat.atomtech.in/ots/payment/refund?merchId=9135&encData=59B80AF255F3D>

Note*-

The URLs mentioned in this document are **UAT**. Merchants are required to consume the **Production link** in a similar manner to conduct transactions.

b) Process Flow Diagram:



Create API Secret Key

Note: Once the merchant is onboarded, a new API secret key will be shared along with other existing keys (Request Key / Response Key / Request Hash Key / Response Hash Key)

If new API secret key needs to be generated, merchant needs to navigate to **API Credentials** module and generate new key. The key is displayed once; the merchant needs to copy and store it securely. The key cannot be shown again after the merchant leaves the page.

Key Generation

1. Log into Merchant Console with 2MFA
2. Navigate to "**API Credentials**"
3. Click "**Generate API Secret Key**"
4. New API Secret key is displayed once — copy and store it securely
5. The key cannot be shown again after the merchant leave the page

Key Rotation

Steps to rotate the key:

1. Log into Merchant Console with 2MFA
2. Click "**Rotate API Secret Key**"
3. New key is displayed once — copy and store it securely
4. Old key remains valid for 48 hours (grace period)
5. After 48 hours, old key stops working automatically
6. Merchant will receive an email confirmation with the grace period deadline

Key Revocation (Emergency)

If The merchant suspect the key is compromised:

1. Log into Console → Click "**Revoke API Secret Key**"
2. Key is immediately invalidated (no 48h grace period)
3. Generate a new key to restore API access

Data Validation

API Secret Key	Format	Type	Example
Total Length → 48 Characters	Always starts with → msk_XXXXXXXXXXXXXXXXXXXXXXX	Mix of uppercase, lowercase, and digits	msk_Xk9mP2qR7vT5wZaB3cD4eF6gH8jK1mN0pQ9rS2tU4v

Create a Bearer Token

To create a Bearer Token, below format is used.

Format	merchantId:apiSecretKey
--------	-------------------------

After this, the data is converted to Base64 format.

Once the data is formatted, it can be used to send in the Bearer token:

Authorization: Bearer <token>

Steps to follow:

- Step 1 — Generate Your API Secret Key
 - Log into Merchant Console (2MFA) → Generate API Secret Key → Store securely.
- Step 2 — Implement Bearer Token
 - Store merchantId and apiSecretKey securely on your server
 - Build credential string: merchantId:apiSecretKey
 - Base64-encode it

- Add Authorization: Bearer <encoded> header

cURL

bash

```
#!/bin/bash

MERCH_ID="546852"

API_SECRET_KEY="msk_Xk9mP2qR7vT5wZaB3cD4eF6gH8jK1mN0pQ9rS2tU4v"

ENC_DATA="0FD2A5551CADDE505215300BC54451B06AFDD07AB4017FC6F4D01C..."

# Set the API path - replace with any affected endpoint from Section 5
API_PATH="/ots/v2/payment/status"

# Build Bearer token
TOKEN=$(echo -n "${MERCH_ID}:${API_SECRET_KEY}" | base64)

# Make request
curl -X POST "${BASE_URL}${API_PATH}?merchId=${MERCH_ID}&encData=${ENC_DATA}" \
  -H "Authorization: Bearer ${TOKEN}"
```

Python

python

```
import base64
import requests

MERCH_ID = "546852"

API_SECRET_KEY = "msk_Xk9mP2qR7vT5wZaB3cD4eF6gH8jK1mN0pQ9rS2tU4v"

# Set the API path - replace with any affected endpoint from Section 5
API_PATH = "/ots/v2/payment/status"
```

```

enc_data = "0FD2A5551CADDE505215300BC54451B06AFDD07AB4017FC6F4D01C..."

# Build Bearer token

credentials = f"{MERCH_ID}:{API_SECRET_KEY}"

token = base64.b64encode(credentials.encode("utf-8")).decode("utf-8")

headers = {"Authorization": f"Bearer {token}"}

response = requests.post(

    f"{BASE_URL}{API_PATH}",

    params={"merchId": MERCH_ID, "encData": enc_data},

    headers=headers,

)

print(response.status_code, response.json())

```

Java

java

```

import java.net.HttpURLConnection;

import java.net.URL;

import java.net.URLEncoder;

import java.nio.charset.StandardCharsets;

import java.util.Base64;

public class OtsApiClient {

    private static final String MERCH_ID = "546852";

    private static final String API_SECRET_KEY =

        "msk_Xk9mP2qR7vT5wZaB3cD4eF6gH8jK1mN0pQ9rS2tU4v";

```



```

public static void main(String[] args) throws Exception {

    // Set the API path – replace with any affected endpoint from Section 5

    String apiPath = "/ots/v2/payment/status";

    String encData = "0FD2A5551CADDE505215300BC54451B06AFDD07AB4017FC6F4D01C...";

    // Build Bearer token

    String credentials = MERCH_ID + ":" + API_SECRET_KEY;

    String token = Base64.getEncoder().encodeToString(
        credentials.getBytes(StandardCharsets.UTF_8));

    String urlStr = BASE_URL + apiPath

        + "?merchId=" + URLEncoder.encode(MERCH_ID, "UTF-8")

        + "&encData=" + URLEncoder.encode(encData, "UTF-8");

    URL url = new URL(urlStr);

    HttpURLConnection conn = (HttpURLConnection) url.openConnection();

    conn.setRequestMethod("POST");

    conn.setRequestProperty("Authorization", "Bearer " + token);

    System.out.println("Response Code: " + conn.getResponseCode());

}
}

```

Node.js

javascript

```

const https = require('https');

const MERCH_ID = '546852';

```

```

const API_SECRET_KEY = 'msk_Xk9mP2qR7vT5wZaB3cD4eF6gH8jKlmN0pQ9rS2tU4v';

// Set the API path - replace with any affected endpoint from Section 5
const API_PATH = '/ots/v2/payment/status';

const encData = '0FD2A5551CADDE505215300BC54451B06AFDD07AB4017FC6F4D01C...';

// Build Bearer token
const token = Buffer.from(`${MERCH_ID}:${API_SECRET_KEY}`).toString('base64');

const queryParams =
  `merchId=${encodeURIComponent(MERCH_ID)}&encData=${encodeURIComponent(encData)}`;

const options = {
  hostname: BASE_HOST,
  path: `${API_PATH}?${queryParams}`,
  method: 'POST',
  headers: { 'Authorization': `Bearer ${token}` },
};

const req = https.request(options, (res) => {
  let data = '';
  res.on('data', (chunk) => { data += chunk; });
  res.on('end', () => { console.log(res.statusCode, data); });
});

req.end();

```

PHP



```
<?php

$merchId      = "546852";

$apiSecretKey = "msk_Xk9mP2qR7vT5wZaB3cD4eF6gH8jK1mN0pQ9rS2tU4v";

// Set the API path - replace with any affected endpoint from Section 5
$apiPath      = "/ots/v2/payment/status";

$encData      = "0FD2A5551CADDE505215300BC54451B06AFDD07AB4017FC6F4D01C...";

// Build Bearer token
$token = base64_encode("{ $merchId }: { $apiSecretKey }");

$url = "{$baseUrl}{$apiPath}?merchId=" . urlencode($merchId)
      . "&encData=" . urlencode($encData);

$ch = curl_init($url);

curl_setopt($ch, CURLOPT_POST, true);

curl_setopt($ch, CURLOPT_HTTPHEADER, ["Authorization: Bearer {$token}"]);

curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

$response = curl_exec($ch);

$httpCode = curl_getinfo($ch, CURLINFO_HTTP_CODE);

curl_close($ch);

echo "HTTP {$httpCode}: {$response}\n";
```

Error Handling

HTTP Code	Error Message	Cause	Resolution
401 Unauthorized	Missing authorization	Authorization header not present	Add Authorization: Bearer <token> header
401 Unauthorized	Invalid token format	Not valid Base64 or missing ':' separator	Verify Base64 encoding of merchantId:apiSecretKey
401 Unauthorized	Invalid credentials	merchantId or apiSecretKey does not match	Check correct key for correct environment
401 Unauthorized	Key expired	API Secret Key expired (past 48h grace)	Generate new key from Console
403 Forbidden	Merchant suspended	Account suspended	Contact NDPSI support

Error Response Format

json

```
{
  "message": "Unauthorized"
}
```

Change under Endpoints

To authenticate using the new API Secret Key, merchants must update the API endpoint URL.

Example:

Current URL

https://payment.atomtech.in/ots/payment/refund

Updated URL

https://payment.atomtech.in/otsv2/payment/refund

2. SINGLE PRODUCT REFUND

a) SINGLE PRODUCT SAMPLE REFUND REQUEST (OPEN DATA):

Request Parameters for Single Product Refund are to be shared in the format illustrated below:

```
{ "payInstrument": {
  "headDetails": {
    "api": "REFUNDINIT",
    "source": "OTS"
  },
}
```

```

"merchDetails": {
  "merchId": 9135,
  "password": "Test@123",
  "merchTxnId": "11234456"
},
"payDetails": {
  "signature":
"7b5e32891aab691be2b7ed174caf8e34552943ba860b00e4e8320f2d2179364e4523f5c7d2459dcc1fd47383d1c
6b802703d7870301a1b4b3cc6f396675732d6",
  "atomTxnId": 11000000223788,
  "totalRefundAmount": 4000.00,
  "txnCurrency": "INR",
  "prodDetails": [
    {
      "prodName": "Mangeshtest",
      "prodRefundAmount": 4000.00,
      "prodRefundId": "189333256"
    }
  ]
}
}
}

```

b) Specifications of API Request:

Parameter Name	Conditional/ Optional/Mandatory	Data Type & Max Length	Sample Value	Content / Remarks
api	Mandatory	String (20)	REFUNDINIT	"REFUNDINIT" for Refund API
merchId	Mandatory	int(15)	9135	Unique ID assigned by NDPS to Merchant
password	Mandatory	String(45)	Test@123	Merchant transaction password
merchTxnId	Mandatory	String(50)	1234567890	Unique transaction ID provided by Merchant
atomTxnId	Mandatory	int(14)	100034566789	Unique transaction ID provided by NDPS
txnCurrency	Mandatory	String(5)	INR	The code of the currency in which the refund is to be made
signature	Mandatory	String(256)	7b5e32891aab691be2b7ed174caf8e34552943ba860b00e4e8320f2d2179364e4523f5c7d2459dcc1fd47383d1c6b802703d7870301a1b4b3cc6f396675732d6	Refer Signature Generation Logic (Pg. 19)

totalRefundAmount	Mandatory	double(12,2)	10.00	Refund Amount (Sum of all refund amount initiated against all products)
prodName	Mandatory	String(50)	NSE	Product ID passed in product name parameter.
prodRefundId	Mandatory	String(45)	234567890	Unique refund ID for Product
prodRefundAmount	Mandatory	double(12,2)	10.00	Product wise Refund amount.

c) SINGLE PRODUCT SAMPLE REQUEST ENCRYPTED DATA:

This is sample encrypted request data as illustrated below:

59B80AF255F3D63E3711DD833E01F98FC098E50B9E09B3C39D0605CC6E7F8AB998E84CBE365D6FD578FB3B770F4B1FF9509C56A3CF7059FF2DA3FEC198AC427B1537DD6D7FE50E54750EE85F1D3713386464291AF0207A9EE9709D6FA4E9D86AB393F5FF9A5EB09070D814EBCB5B0E6C37752D1182454B52C2FFC4477E01ABC1A7DCF565234854F774218D6DCEB109331026E2E26C102B451CDFAAE85BAACE6DF5966319E2D550CF8177D10AB089BBDDF9EC391C2A7DAC3C5D6471C3056FEDAD11DAC3A6CEABBE12D13A3F9C0EAC1CB1715696DDDC740D0B2E66A777F87CDC7533A69E98D018908AAE19A6734482B862EC31EDA59F60FEAC1BA27FCB59E8A284BC646CFFCB2414D3ACF695A509612DA251DB84B82D2AEB3CA57950F456512D379DB0667F48D9DAE85B0078B576C02F57F8B1D1D7E718F2F7604B8C9B3CBC9B50D6AA7EE4530C63680117ADEC402E216729DD663F1CD825DF24561B7C43163A698A219E7737A20C82C1AFAC550829ED58F2D625FCB1A04EBC4A7CCB334A3491B04B2592C5FD225327916945C190620BE3E50FA64E6834ED4A0917A61B5C4E344C74CA82AA05D7AF32ABC0220D800D38F5FEA734157668C5AC342A0A8CDC96F1044229A96BCCEF856AA84D628CA21EA627A382E01E0F290DF757FD3FA440A86101F5092E2093734F284D78FCA8301D5ABCB3BC215020FC653BC1BF90064E31D0B50A7AD51962F9EA58C483B70A0D5897961C2D33A9C49DB65D3AB18BC87FC4CBE8E33B6A53ABA5878CEE08F1ADA1893FC837D0D0B16DFFF6BD39BFBDF784771D5F356FF779288ECADE20D19F7B3BABC323028B01F86413A6DD71548A25F73F37F7259AB67E07EB2BA2690F0BF6BAA2FA59FD04E5C334501338C05B2C4802E92E27B4DC1D5A0F226EBE4611D4628112AC97C3CEF0CA3A3C46FCFAF71BB3A961F7E9

a) SINGLE PRODUCT SAMPLE REFUND REQUEST (DECRYPTED):

<https://caller.atomtech.in/ots/payment/refund?merchId=9135&encData=59B80AF255F3D63E3711DD833E01F98FC098E50B9E09B3C39D0605CC6E7F8AB998E84CBE365D6FD578FB3B770F4B1FF9509C56A3CF7059FF2DA3FEC198AC427B1537DD6D7FE50E54750EE85F1D3713386464291AF0207A9EE9709D6FA4E9D86AB393F5FF9A5EB09070D814EBCB5B0E6C37752D1182454B52C2FFC4477E01ABC1A7DCF565234854F774218D6DCEB109331026E2E26C102B451CDFAAE85BAACE6DF5966319E2D550CF8177D10AB089BBDDF9EC391C2A7DAC3C5D6471C3056FEDAD11DAC3A6CEABBE12D13A3F9C0EAC1CB1715696DDDC740D0B2E66A777F87CDC7533A69E98D018908AAE19A6734482B862EC31EDA59F60FEAC1BA27FCB59E8A284BC646CFFCB2414D3ACF695A509612DA251DB84B82D2AEB3CA57950F456512D379DB0667F48D9DAE85B0078B576C02F57F8B1D1D7E718F2F7604B8C9B3CBC9B50D6AA7EE4530C63680117ADEC402E216729DD663F1CD825DF24561B7C43163A698A219E7737A20C82C1AFAC550829ED58F2D625FCB1A04EBC4A7CCB334A3491B04B2592C5FD225327916945C190620BE3E50FA64E6834ED4A0917A61B5C4E344C74CA82AA05D7AF32ABC0220D800D38F5FEA734157668C5AC342A0A8CDC96F1044229A96BCCEF856AA84D628CA21EA627A382E01E0F290DF757FD3FA440A86101F5092E2093734F284D78FCA8301D5ABCB3BC215020FC653BC1BF90064E31D0B50A7AD51962F9EA58C483B70A>

[0D5897961C2D33A9C49DB65D3AB18BC87FC4CBE8E33B6A53ABA5878CEE08F1ADA1893FC837D0D0B16DFFF6BD39BFBDF784771D5F356FF779288ECADE20D19F7B3BABC323028B01F86413A6DD71548A25F73F7F7259AB67E07EB2BA2690F0BF6BAA2FA59FD04E5C334501338C05B2C4802E92E27B4DC1D5A0F226EBE4611D4628112AC97C3CEF0CA3A3C46FCAF71BB3A961F7E9](#)

b) SINGLE PRODUCT SAMPLE REFUND RESPONSE DATA (ENCRYPTED):

This is sample encrypted response as illustrated below:

encData=F5140AF9DC1B3DB7AFA300D9675EE72A23B8826A6ED4001BD3E1B4760A58413789AF19B09B7AD993AEB53212F4D1933B49A281C3D8C99DF6BA0EDD1D629D2B4230718B7BC8E7453DC06D920D6373AA85241219116DF2C342968F45FE2CE4018DD13D4F5F7FAE2CD11D81C29CE29A5E5D9DDA03A4453D807C80A12DA8C899759A43112C727BA9644F4B56657B7916ED6F9BCF5B8A72F7E09721BE5E1A5CECF33DBAFA2CBB7B3DD3ED57207D8826D8C581350E66CBF35F1ABC8303CB0A623A903AC13462E95ED233FCE82242DEF8B3ED70EC7BADAA7B042CA761D815597A4DE4435450F62E80AEC84003AFA23CF29C7CCD506020BD01E2DF957E3A8757A5898785FEAA1473E8F67D0970AF447E11E27C3F11C8BCB58DFD02D629E0A059B27218C2E8859F706B9241779C6BE5279DACB9D48F76CB76A0E390352EF2BF5668385EBE1D979751FB0A6A761198D0ADD977DDDC017F0A16B7696FF7C77C39BA5E62B28E7B9D2852E93B8917D4070E91C0F3A6E96F0B98A0B6CC502C23C435E58925D6C61C773445277B2F5A6CCD77B1DC6550CDA8D135AE1F5E78685D4AE1C958444C146999C6A44FD328F7B0F90E7582AB20F7&merchId=9135

c) SINGLE PRODUCT SAMPLE RESPONSE DECRYPTED (OPEN DATA):

Response Parameters are obtained in the format illustrated below:

```
{
  "payInstrument": {
    "payDetails": {
      "atomTxnId": 11000000223788,
      "prodDetails": [
        {
          "prodName": "Mangeshtest",
          "prodRefundAmount": 4000,
          "prodRefundId": "189333256",
          "refundTxnId": 1519,
          "prodDescription": "Full Refund Initiated Successfully",
          "prodStatusCode": "OTS0000"
        }
      ],
      "totalRefundAmount": 4000,
      "txnCurrency": "INR"
    },
    "responseDetails": {
      "statusCode": "OTS0000",
      "message": "SUCCESS",
      "description": "Full Refund Initiated Successfully"
    }
  }
}
```

d) Specifications of API Response:

Parameter Name	Conditional/ Optional/Mandatory	Data Type & Max Length	Sample Value	Content / Remarks
atomTxnId	Mandatory	int (14)	100034566789	Unique transaction ID provided by NDPS
totalRefundAmount	Mandatory	double(12,2)	10.00	Refund Initiated amount (Sum of all refund amount initiated against all products)
txnCurrency	Mandatory	String(5)	INR	The code of the currency in which the refund is made
prodName	Mandatory	String(50)	NSE	Product ID passed in the product name parameter
prodRefundAmount	Mandatory	double(12,2)	10.00	Product wise Refund amount
prodRefundId	Mandatory	String(45)	234567890	Unique Refund ID for Product
refundTxnId	Mandatory	int(8)	12345678	Unique Refund Transaction id
prodDescription	Mandatory	String(50)	Refund Initiated Successfully	Product Description
prodStatusCode	Mandatory	String(7)	OTS0000	Product Status Code
statusCode	Mandatory	String(7)	OTS0000	Status code
message	Mandatory	String(10)	SUCCESS	Status Message
description	Mandatory	String(50)	Refund Initiated Successfully	Message description

3. SINGLE PRODUCT PARTIAL REFUND

a) SINGLE PRODUCT SAMPLE PARTIAL REFUND REQUEST (OPEN DATA):

```
{  
  "payInstrument": {  
    "headDetails": {  
      "api": "REFUNDINIT",
```



```

    "source": "OTS"
  },
  "merchDetails": {
    "merchId": 9135,
    "password": "Test@123",
    "merchTxnId": "7777777"
  },
  "payDetails": {
    "signature":
    "419e55e58cf5da634a369f66c48c8b1e6eb5635883c1596f3aeb9ea7b925313c9938b57f84b62721c3975f978c2
    2be8b0c1e1921b368fa57a6c0e45073ae76d8",
    "atomTxnId": 11000000222715,
    "totalRefundAmount": 10.00,
    "txnCurrency": "INR",
    "prodDetails": [
      {
        "prodName": "Mangeshtest",
        "prodRefundAmount": 10.00,
        "prodRefundId": "567471258"
      }
    ]
  }
}

```

b) SINGLE PRODUCT SAMPLE PARTIAL REFUND REQUEST DATA (ENCRYPTED):-

merchId=9135&encData=59B80AF255F3D63E3711DD833E01F98FC098E50B9E09B3C39D0605CC6E7F8
 AB998E84CBE365D6FD578FB3B770F4B1FF9509C56A3CF7059FF2DA3FEC198AC427B1537DD6D7FE
 50E54750EE85F1D3713386464291AF0207A9EE9709D6FA4E9D86AB393F5FF9A5EB09070D814EBCB5
 B0E6C37752D1182454B52C2FFC4477E01ABC1A7DCF565234854F774218D6DCEB109331026E2E26C1
 02B451CDFAAE85BAACE6DF5966319E2D550CF8177D10AB089BBDDF9EC391C2A7DAC3C5D6471C
 3056FEDAD14283CF702F2D290F1E33C1A30C427DC28B2B048C072512FAA46563A8DC6A8CE2B1576
 6B49563AE0812A2A8F071C43846FD4078CDC9419BBCA2218151AE3B6ED63B8347E4A3CCD73081B0
 5A8FA9844FFB73029589AF18FF041F356CF0014F660CD200AC30785743BED208F37A096F8A92F3B8B
 AFB80249DDED4081AA15D63C13E01A1D3FFD59829AE596293A0A2902F1B56579435E32141F79B0D
 77051D5765F6656D7CF4EE410F5B476FB1A1C8AE3101B2E77A30288C42E33ACDD077E8541E78A1B
 CFCA5D8181402BBE847BDDCD9186980CD954510F3FAFDD863672F4B730979C0B19333FFA0E0D6E5
 40B08E7BE54DA7090E60A32372414FC0B545B4002240E272B27C986F64138CF906165ED8979B3BB6F
 CFD175157CEE401079879ED8DF8544F1518C6BEE5E698B6A310FD77EF49D333D340C478603DC3365
 5E3A370560D5A3DBFABB12850B7170522548819AC2E6843F61D84D91951C9FDE643747C961D41E77
 2B944A5A2DB7F90F0E620C33738828228F286849D6C58EAD68AC63A345AF21C8E4F3A91AE669BD0
 F320EF53E96B6C53563BE163E482D8802FB0E20E0AD34E37415777ECFA073FAC6001BFDB1D4D65C
 84503271A739DE337B4E60BAFB70614646F8FFE35709F62D5929BF380EED82

c) SINGLE PRODUCT SAMPLE PARTIAL REFUND RESPONSE (ENCRYPTED):-

encData=F5140AF9DC1B3DB7AFA300D9675EE72A23B8826A6ED4001BD3E1B4760A58413789AF19B09B7AD993AEB53212F4D1933BFA19D7A948106E118F682B2646D9D56D43B5F5BE3A7078BE7D349DC94BD8BA6A3E0EA82774C2B0E5EA3C7E509D0CC8FD808E5472806EEFD39235C187773C28C6F338DE93CD3586DBF0CF7835720C3DE3A41B25E4E6A73AEEB68254D92367CA3EE37942190CF73FB674C0262427CB3F2DC99391B5FF7D932A17346AC5F2F6E6B4C69E2EC2536A1B096979464A8BA7D64B1D3F889A2CC720CBC479FFA749ECCF3DABF7D5DD20CA12FFCA9BCDB5C1DAE22A320F20381E2A2C118ACA29E54F1B6C10740AB8B88F713B3FFD977E18AFB87797BBB56944704DCA98D5BB90E7506A4754E722905B6E40A9AA39DBD1E0E7A14D5DA0653C5C76F2E7FB5AA82C8A14AADEB5EFAF0DDF40C19D392C6A72D82727DEBC4A358727A2019519270A2764895CC993BBA41755DAE112C802AC18A2394D5459E51A0EA83F7566A5DCD6F6BCABF73F20195BDF525532A796E40D1BC250DCC4056BD1018526106C1FC29CDE1B19242C65720AB1067ECB3C920670B2192518E18DE94DA8BE7FDCCE177CD694628E96BAB5&merchId=9135

d) SINGLE PRODUCT SAMPLE PARTIAL REFUND RESPONSE (DECRYPTED):-

Response Parameters are obtained in the format illustrated below:

```
{
  "payInstrument": {
    "payDetails": {
      "atomTxnId": 11000000222715,
      "prodDetails": [
        {
          "prodName": "Mangeshtest",
          "prodRefundAmount": 10,
          "prodRefundId": "567471258",
          "refundTxnId": 1515,
          "prodDescription": "Partial Refund Initiated Successfully",
          "prodStatusCode": "OTS0001"
        }
      ],
      "totalRefundAmount": 10,
      "txnCurrency": "INR"
    },
    "responseDetails": {
      "statusCode": "OTS0001",
      "message": "SUCCESS",
      "description": "Partial Refund Initiated Successfully"
    }
  }
}
```

4. MULTI PRODUCT REFUND

a) MULTI PRODUCT SAMPLE REFUND REQUEST (OPEN DATA):-

Request parameters for Multi Product Refund are to be shared in the format illustrated below:

```

{
  "payInstrument": {
    "headDetails": {
      "api": "REFUNDINIT",
      "source": "OTS"
    },
    "merchDetails": {
      "merchId": 9135,
      "password": "Test@123",
      "merchTxnId": "TSTTXN543"
    },
    "payDetails": {
      "atomTxnId": 11000000229212,
      "txnCurrency": "INR",
      "signature":
"9cfac773f90fa8aad3e196293ffd0366ee90f987851e3b1762383b69842fb5be4c689ecbd406532f0aafc83b6047fae2a
530de2d111d9b82b079c8c51d7a61b6",
      "totalRefundAmount": 23.00,
      "prodDetails": [
        {
          "prodName": "DHARAM_TEST",
          "prodRefundId": "666645682",
          "prodRefundAmount": 12.00
        },
        {
          "prodName": "Mangeshtest",
          "prodRefundId": "666645683",
          "prodRefundAmount": 11.00
        }
      ]
    }
  }
}

```

b) MULTI PRODUCT SAMPLE REFUND REQUEST DATA (ENCRYPTED):-

merchId=9135&encData=59B80AF255F3D63E3711DD833E01F98FC098E50B9E09B3C39D0605CC6E7F8
AB998E84CBE365D6FD578FB3B770F4B1FF9509C56A3CF7059FF2DA3FEC198AC427B1537DD6D7FE
50E54750EE85F1D3713386464291AF0207A9EE9709D6FA4E9D86A893FF1215CC21E386967AE0C0E69
1D778A078BCC4616B702E3C7A86D907AA1C27F8666CCC143CBF0D6964C73C47E966697C18D86B6E
0332AC21724DAA395846F01D87853684385336EDE0C5B26C8F1B0C931AE006D97F42732DC16663217
5D892DEF7F8C2B91F043C96562E508992FC2D1C0DB3910EA9F774D92C7B5960B87C16098B54392F44
ED613096F24D060A60C4A611EB557DAC20D39DF9CEA26B6FAA7A95D1CF24E9E0E212D3A95AA84
3E6D388D3E537AB9EE0D2AEC83F14E6A0583E5E74B3150F0A0802885AF3CB208A9C8A55D15B950C
4BD43AFB03E85D35A93D1003958B8B1A75E6E36AE326100768D07E8DF458199911A89C3891CC965C
F4B9E78E0F262F1E30650DF0942F766172BECB3D928BA2C44C4C4CC82FF4A4E60C3D66B81DA2A51
FEB2D3C74E486CA6E3DF9496DEE04BA4E45CDB95B5DFF0B9A3A11FE59C869F8E86DF4F2FF15FB
C9A1CC0B058F51F1DA5C41258A390F7FECC052CBD22E13025924521864A42A6DC80B0B00C5F94C0
6BA40501C078D16F5951E06C58D124D10D6A4F5FE9CBB062B8EC9B2861AB6EC63AEF16A3F2EC3E

9A15820C69B1FA19B7B789936FBE3D1F83B6D970B9D02C87AC0B3EAECB06D62B90DA0F2CA36F5
ACBAA5FC98ADA3D6FFD8C9EEAEE55434DBA503C6D06A9AD648349BB32A4A62C366662C2AF58
DBDF6A6D26DAD5760C86A28B089A10AD7BC6149B7C34B5F426342BC1775085ABB9D5CE64C6B01
7DC976EF66AC1A03FDD89DB1C8689D7B84AD0E1EE745760FF626517E654822008B958A37098179D5
37BC5FB2F158D2CCB51D29FD6373C717891232198AFE5D90BECBF2F3C6836519BB9F596FFF76DA0
9123EDB5D4E002FDD6ADF616B8FAB10D69E660A0C07DA280004557DC0E5E96528E51DF075D45545
E93358193312959EEAF0B117C92E18B83F6EE4DEAE2BB7291BC6EAAC61D4F16E2EC3194EAB9B483
5FE547E36F2A9D3D5778AAF232C3687F2463CD9F042E5C49F64CC7D4281FFAEB31290FC641C1ECD
E2427300B557E8A2EAC1DB7E20BB581A6C5770B1E9DFFDC1CA909C0F94A5BFAE97C2FB73754C4E
3D91FC6FE763A032D925F6AD415D986793D000B3FB5942B5F0C6197B4BC72C3750B527681CAE7453
374CC68DB92155FCA92DFBCDF26E8F63B8CA390FC63A80A76FE20FC1524D412187115789689C496
A1D30A2980ABC6A84F4D045B63138B04B87AC1924F878A09

c) Specifications of API Request:

Parameter Name	Conditional/ Optional/Mandatory	Data Type & Max Length	Sample Value	Content / Remarks
api	Mandatory	String (20)	REFUNDINIT	"REFUNDINIT" for Refund API
merchId	Mandatory	int(15)	9135	Unique ID assign by NDPS to Merchant
password	Mandatory	String(45)	Test@123	Merchant wise password
merchTxnId	Mandatory	String(50)	1234567890	Unique Transaction ID provided by Merchant
atomTxnId	Mandatory	int (14)	100034566789	Unique transaction id provided by NDPS
txnCurrency	Mandatory	String(5)	INR	The code of the currency in which the refund is to be made
signature	Mandatory	String(256)	eced0d634 6fa66376d 40cbc1c5a8 12c5f60fbc 9c2fe756c4 59f5e39c34 55ba7ebb9 053cd3d3b	Refer Signature Generation Logic (Pg. 19)
totalRefundAmount	Mandatory	double(12,2)	10.00	Refund Amount (Sum of all refund amount initiated against all products)
prodName	Mandatory	String(50)	NSE	Product ID passed in the product name parameter.

prodRefundId	Mandatory	String(45)	234567890	Unique Refund ID for product provided by the Merchant
prodRefundAmount	Mandatory	double(12,2)	10.00	Product wise Refund Amount.

d) MULTI PRODUCT SAMPLE REFUND RESPONSE DATA (ENCRYPTED):

This is sample encrypted response as illustrated below: -

encData=F5140AF9DC1B3DB7AFA300D9675EE72A23B8826A6ED4001BD3E1B4760A58413789AF19B09B7AD993AEB53212F4D1933B50EB242EDE540C25CDD6D22E38AA99A36408BFC63741EA935431C3ED8664FB7797C62B960690030B2F89A74D4FB06C9BA1C4C1AA850ED0B62A38F4CD8A5D98941CE6EB59A06641798D808657EE46851C2A9E5AF20FBDA7288BC114B5FA275938D18C83A72C71AFE7BDC622BA51DC6DEFE3630BD4E09131DEA1643EB42CD9C308D41B3A80D3E91CDD35396B0CC5B515DFA6FCD3054F08F5F4BA8335AB140A6F886161CD7C6BED07011E9EE159CBEC8B3658377E2D1E49257AEFB45D779C4C7D2963AEA53223BF5E7CCB981CCA824D71E6811336FC135AB3815CB700218A1EB402B274B17E24B4E602E04BE39BB7D74A669164CD8C9CC6FB6AC85A864A5D518AF4C223244031386BC95946834268FB691C75568D9F418D00182FB39291ECDB7C3640FEACED4A0D660CBCBDCCC3B43E7B7D4AC09AEEBFD3FA47BBA933D5426949E12B02576D4FCBA547C1F080CDC8595C33C545EE011CA9D859F8EC07F29D7F7438B5C113C5103F65B369DE360D5F4BC9D79DF55D002396677178190F5971D7F324A2188DFFD388AF8306577C9168F38551A94BA1DCE50EFA107A794960D47683320C73AF8A25B4BA544472EC146F9812E1409A52E60D435301130AD2C9A961D23961AE36DB2EB61D6D4DD20787D6AF4622029B0D5684C20EE0C10FF632A9BBF53D51AA50ACF5D797BE7D52BC27107D93406469BCA28A1005E26CAB3F6F262BEAE9E6D2594948A14D865B53EE3908B7B0DB40B90260F2EA68B3E026B952208F5847E6BDB693CD1593C87B78B9C6149F4022&merchId=9135

e) MULTI PRODUCT SAMPLE DECRYPTED REFUND RESPONSE (OPEN DATA):

Response parameters for Multi Product Refund are obtained in the format illustrated below:

```
{
  "payInstrument": {
    "payDetails": {
      "atomTxnId": 11000000229212,
      "prodDetails": [
        {
          "prodName": "DHARAM_TEST",
          "prodRefundAmount": 12.00,
          "prodRefundId": "666645682",
          "refundTxnId": 1569,
          "prodDescription": "Full Refund Initiated Successfully",
          "prodStatusCode": "OTS0000"
        }
      ],
    },
  },
}
```

```

        "prodName": "Mangeshtest",
        "prodRefundAmount": 11.00,
        "prodRefundId": "666645683",
        "refundTxnId": 1570,
        "prodDescription": "Full Refund Initiated Successfully",
        "prodStatusCode": "OTS0000"
    }
},
    "totalRefundAmount": 23.00,
    "txnCurrency": "INR"
},
    "responseDetails": {
        "statusCode": "OTS0000",
        "message": "SUCCESS",
        "description": "Full Refund Initiated Successfully"
    }
}
}

```

f) Specifications of API Response:

Parameter Name	Conditional/ Optional/Mandatory	Data Type & Max Length	Sample Value	Content / Remarks
atomTxnId	Mandatory	int (14)	100034566789	Unique Transaction ID provided by NDPS
totalRefundAmount	Mandatory	double(12,2)	10.00	Refund Initiated Amount (Sum of all refund amount initiated against all products)
txnCurrency	Mandatory	String(5)	INR	The code of the currency in which the partial refund is made
prodName	Mandatory	String(50)	NSE	Product ID passed in the product name parameter.
prodRefundAmount	Mandatory	double(12,2)	10.00	Product wise Refund Amount.
prodRefundId	Mandatory	String(45)	234567890	Unique Refund ID for Product provided by the Merchant
refundTxnId	Mandatory	int(8)	12345678	Unique Refund Transaction ID
prodDescription	Mandatory	String(50)	Refund Initiated Successfully	Product Description
prodStatusCode	Mandatory	String(7)	OTS0000	Product Status Code

statusCode	Mandatory	String(7)	OTS0000	Status Code
message	Mandatory	String(10)	SUCCESS	Status Message
description	Mandatory	String(50)	Refund Initiated Successfully	Message description

5. MULTI PRODUCT PARTIAL REFUND

a) MULTI PRODUCT SAMPLE PARTIAL REFUND REQUEST DATA (Open Data):-

Request parameters for Multi Product Partial Refund are to be shared in the format illustrated below:

```
{
  "payInstrument": {
    "headDetails": {
      "api": "REFUNDINIT",
      "source": "OTS"
    },
    "merchDetails": {
      "merchId": 9135,
      "password": "Test@123",
      "merchTxnId": "TSTXN6000"
    },
    "payDetails": {
      "atomTxnId": 11000000229597,
      "txnCurrency": "INR",
      "signature":
"fb0798ae1602bc0d395b7a12e8f08ab5d0ec64e90a2b9e7f09ed3db847ac86820edb1a0d557c51aec2e0166fae97d4aa
966a6f0d8512fd454e4deb906959c1bf",
      "totalRefundAmount": 40.00,
      "prodDetails": [
        {
          "prodName": "DHARAM_TEST",
          "prodRefundId": "666645679",
          "prodRefundAmount": 30.00
        },
        {
          "prodName": "Mangeshtest",
          "prodRefundId": "666645678",
          "prodRefundAmount": 10.00
        }
      ]
    }
  }
}
```

b) MULTI PRODUCT SAMPLE PARTIAL REFUND REQUEST DATA (ENCRYPTED):-

merchId=9135&encData=59B80AF255F3D63E3711DD833E01F98FC098E50B9E09B3C39D0605CC6E7F8
AB998E84CBE365D6FD578FB3B770F4B1FF9509C56A3CF7059FF2DA3FEC198AC427B1537DD6D7FE
50E54750EE85F1D3713386464291AF0207A9EE9709D6FA4E9D86AB393F5FF9A5EB09070D814EBCB5
B0E6C37752D1182454B52C2FFC4477E01ABC1A7DCF565234854F774218D6DCEB109331026E2E26C1
02B451CDFAAE85BAACE6DF5966319E2D550CF8177D10AB089BBDDF9EC391C2A7DAC3C5D6471C
3056FEDAD73D722B28F4FAE3F712BDA8355F929C0A0398A554DEB1605D0E1C33137CA40209F74CB
44A48238F436D20A97783FF04EB78C7A78D8530A12CAD276014BAA7C4C267FC46F9F126E624E383E
B8EE034592FA77CEE45CCDFCA5DFAE80C60CF2736EB948935D54C94ABC95CCB41017555EB4C317
C5B4F2A4A9C4BEC2A2EC2324AAF8662AAD58B5ED84D4E7EC13E01A08F5E89515718E26B30D26C2
9FB5061D6018502C6505A6FEA03F1176F12F80B1E873111E5643E4E07BC95597DE90E12D90DF365873
C570821AC5604133CE5FE7CF1C91FD61823B7EC410E6CCBDC98265C782B61A4510CDBF3190A29C3
9877F53DF05907BCA9689F2D62A06CBDBF4940EE57A2C1F344685C74B1C5EEF50301B202E1808033
B3F3E5FE2FD5335779B7AA63A6278716E099A6C3BE8787217F73CFCBEA66D922370F9B2981CDA4B
6C392C2A0D0B3813C4CD44C14BE7222766847478D2B927D8498B972102258A2BBF75965DB8A50BB8
070F00A2754DEFEE31FA70BED9460F4532AD4CCA0EB70554FF052BF36EAD0178140426299BE247B0
7B329E77D5DB424C30D923CE2DDB26377D4D9FD83A4C9ACDE026A1332C1F740F18606A0F651F189
A6B976D92D93519877AE2E0F90B9ACE9843679471290E429BC891EA4E3A24FFF8D77CC64D5AE9B8
2B5D8B06E7228A3C8FFAC34B43E0DDAC28F4067BD0D5241DBA5B4F3F2E4B0871160262352831D92
83784D59887CA3FA2B73A752F40759B205BC7621997E18AF60FAB7AAB20AC51845DA6096463120E4
A23711B9E0E217DD436439DC1DA66600C17CFB821FDDD64BE17F6C68A574A25EF02699EB766C9E3
84

c) MULTI PRODUCT SAMPLE PARTIAL REFUND RESPONSE DATA (ENCRYPTED):-

encData=F5140AF9DC1B3DB7AFA300D9675EE72A23B8826A6ED4001BD3E1B4760A58413789AF19B0
9B7AD993AEB53212F4D1933BFD9F939242EE65C1D846269F942D17F7CB783A5B5157D5FB5CCD05B3
A88FFDAF9FD5476DD2A195D965EE91C0E2B248A1F0E3BA63CE921E489A963B1C83052AB80DC2B3
35FBBB6B31A6D4C02670DFD8BD732829737AB583D73E4FFA13803E31AE5D97B0DA243378A70208F
3D23B7B64505F4C39F6E6AC1EC0FD3D5417F82FBC83AD50406308E5F137F526A6C171AF773649AFF
AD609A4D2B31588AD11E51B8EC5889320A6BCE2D9F0EF6D2076782C96737CA9CBCA4848BC1826E
70444952050666DDCA44F629BA5C8E402F394C6A88BB41E2500994C5421E85DE48DCF5D00882D26C
2D084C9E474144E8873080C15285AAFC561003D5FC75B1A5DC62632A9595CE0E0178217BF46F8D48
B327449B52F925D67E747D89B6FA41E21CADAAEDC32A82A4AF46FD19F3C21D9BB8F50CB6021343
41BB157720FB256FFF1CEFC9E0CE1D027687955249EF6D14665C7FCA6F6A4C2663985776C202274FF
AAD145F738B1083E38DC2F06FFE0F47AAFE0C553FBDCEDC2EE2DB3A6044334A824DEDA95B7B56
CA8ED3A940965955F90757AA9A234E07F03E97617FA10E2D8186E838FAA159C1C803C5C4ECDD7A0
53D0DD40C82B0AD152CB897E53DED5F1256F6DA03FBB8C8E78CF1DEED1340DD1BA20D40264748
1650B348DCE45E4ABEDFBE8D604EBE81160C92F3344D75CCC94E72942CAE762FF13F2207E942768
9CE51590609A4B1CB16195CB7B0D7CF990338BA2A8AE3F97D311D5AC61DA70828AB8244D132F089
D160BD96278F730A3196A1DC6A82BD404F87814B0&merchId=9135

d) MULTI PRODUCT SAMPLE DECRYPTED PARTIAL REFUND RESPONSE (DECRYPTED):-

Response parameters for Multi Product Partial Refund are obtained in the format illustrated below:


```

{
  "payInstrument": {
    "payDetails": {
      "atomTxnId": 11000000229597,
      "prodDetails": [
        {
          "prodName": "DHARAM_TEST",
          "prodRefundAmount": 30,
          "prodRefundId": "666645679",
          "refundTxnId": 1565,
          "prodDescription": "Full Refund Initiated Successfully",
          "prodStatusCode": "OTS0000"
        },
        {
          "prodName": "Mangeshtest",
          "prodRefundAmount": 10,
          "prodRefundId": "666645678",
          "refundTxnId": 1566,
          "prodDescription": "Full Refund Initiated Successfully",
          "prodStatusCode": "OTS0000"
        }
      ],
      "totalRefundAmount": 40,
      "txnCurrency": "INR"
    },
    "responseDetails": {
      "statusCode": "OTS0001",
      "message": "SUCCESS",
      "description": "Partial Refund Initiated Successfully"
    }
  }
}

```

6. AES ENCRYPTION LOGIC

- Request and response will be encrypted & decrypted using AES-256 algorithm (refer below)
- The following KEY are to be used for UAT:

MerchId	encResKey	encReqKey
9135	58BE879B7DD635698764745511C704AB	7813E3E5E93548B096675AC27FE2C850

a) AES Encryption Java Code:

```

import java.util.logging.Logger;
import javax.crypto.Cipher;
import javax.crypto.SecretKey;
import javax.crypto.SecretKeyFactory;

```

```

import javax.crypto.spec.IvParameterSpec;
import javax.crypto.spec.PBEKeySpec;
import javax.crypto.spec.SecretKeySpec;

public class AtomEncryption {
    static Logger log = Logger.getLogger(AtomEncryption.class.getName());

    private static int pswdIterations = 65536;
    private static int keySize = 256;
    private static final byte[] ivBytes = {
        0,1,2,3,4,5,6,7,8,9,10,11,12,13,14,15
    };

    public static String encrypt(String plainText, String key) {
        try {
            byte[] saltBytes = key.getBytes("UTF-8");

            SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA512");
            PBEKeySpec spec = new PBEKeySpec(key.toCharArray(), saltBytes, pswdIterations, keySize);

            SecretKey secretKey = factory.generateSecret(spec);
            SecretKeySpec secret = new SecretKeySpec(secretKey.getEncoded(), "AES");

            IvParameterSpec locallyIvParameterSpec = new IvParameterSpec(ivBytes);
            Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
            cipher.init(1, secret, locallyIvParameterSpec);

            byte[] encryptedTextBytes = cipher.doFinal(plainText.getBytes("UTF-8"));

            return byteToHex(encryptedTextBytes);
        } catch (Exception e) {
            log.info("Exception while encrypting data:" + e.toString());
        }

        return null;
    }

    public static String decrypt(String encryptedText, String key) {
        try {
            byte[] saltBytes = key.getBytes("UTF-8");
            byte[] encryptedTextBytes = hex2ByteArray(encryptedText);

            SecretKeyFactory factory = SecretKeyFactory.getInstance("PBKDF2WithHmacSHA512");
            PBEKeySpec spec = new PBEKeySpec(key.toCharArray(), saltBytes, pswdIterations, keySize);

            SecretKey secretKey = factory.generateSecret(spec);
            SecretKeySpec secret = new SecretKeySpec(secretKey.getEncoded(), "AES");

            IvParameterSpec locallyIvParameterSpec = new IvParameterSpec(ivBytes);
            Cipher cipher = Cipher.getInstance("AES/CBC/PKCS5Padding");
            cipher.init(2, secret, locallyIvParameterSpec);

```

```

        byte[] decryptedTextBytes = (byte[]) null;
        decryptedTextBytes = cipher.doFinal(encryptedTextBytes);

        return new String(decryptedTextBytes);
    } catch (Exception e) {
        log.info("Exception while decrypting data:" + e.toString());
    }

    return null;
}

private static String byteToHex(byte[] byData) {
    StringBuffer sb = new StringBuffer(byData.length * 2);

    for (int i = 0; i < byData.length; ++i) {
        int v = byData[i] & 0xFF;
        if (v < 16)
            sb.append('0');
        sb.append(Integer.toHexString(v));
    }

    return sb.toString().toUpperCase();
}

private static byte[] hex2ByteArray(String sHexData) {
    byte[] rawData = new byte[sHexData.length() / 2];
    for (int i = 0; i < rawData.length; ++i) {
        int index = i * 2;
        int v = Integer.parseInt(sHexData.substring(index, index + 2), 16);
        rawData[i] = (byte) v;
    }

    return rawData;
}

public static void main(String[] args) {

    try {

        String encryptedData = AtomEncryption.encrypt("1235", "ASWKLSLLFS4sd4g4gsdg");
        System.out.println("encryptedData : " + encryptedData);
    } catch (Exception e) {
        // TODO: handle exception
    }

}
}

```

7. SIGNATURE GENERATION LOGIC

- For above mentioned Refund API's request and their response signature fields to be generated using the following shared Hashing code.

- The signature type is HMACSHA512.
- Signature generation sequence - [merchId + password + merchTxnId + total amount + txnCurrency + api]

a) Sample Request

- [9132Test@123OTS77SL3.00INRREFUNDINIT](#)

b) Generated Signature

fb0798ae1602bc0d395b7a12e8f08ab5d0ec64e90a2b9e7f09ed3db847ac86820edb1a0d557c51aec2e0166fae97
d4aa966a6f0d8512fd454e4deb906959c1bf

c) Hash Keys

Below are the merchant id, request & response hash key –

MerchId	reqHashKey	respHashKey
9135	KEY1234567234	KERESPY1234567234

d) Encryption Key

- Below are the merchant id, request & response encryption key –

MerchId	encReqKey	encResKey
9135	58BE879B7DD635698764745511C704AB	7813E3E5E93548B096675AC27FE2C850

e) Signature Generation Java Code:

```
import java.io.PrintStream;
import java.io.UnsupportedEncodingException;
import java.security.InvalidKeyException;
import java.security.Key;
import java.security.NoSuchAlgorithmException;
import javax.crypto.Mac;
import javax.crypto.spec.SecretKeySpec;

public class AtomSignature {
    public static String generateSignature(String hashKey, String[] param) {
        String resp = null;
```

```

        StringBuilder sb = new StringBuilder();
        for (String s: param) {
            sb.append(s);
        }

        try {
            System.out.println("String =" + sb.toString());
            resp = byteToHexString(encodeWithHMACSHA2(sb.toString(), hashKey));
        } catch (Exception e) {
            System.out.println("Unable to encocd value with key :"+ hashKey + " and input :"+
sb.toString());
            e.printStackTrace();
        }
        return resp;
    }
    private static byte[] encodeWithHMACSHA2(String text, String keyString)
throws NoSuchAlgorithmException, InvalidKeyException, UnsupportedEncodingException {
        Key sk = new SecretKeySpec(keyString.getBytes("UTF-8"), "HMACSHA512");
        Mac mac = Mac.getInstance(sk.getAlgorithm());
        mac.init(sk);
        byte[] hmac = mac.doFinal(text.getBytes("UTF-8"));

        return hmac;
    }
    public static String byteToHexString(byte byData[]) {
        StringBuilder sb = new StringBuilder(byData.length * 2);

        for (int i = 0; i < byData.length; i++) {
            int v = byData[i] & 0xff;
            if (v < 16)
                sb.append('0');
            sb.append(Integer.toHexString(v));
        }
        return sb.toString();
    }
}

```

8. UAT ENVIRONMENT DETAILS

The UAT environment details are as follows:

13.127.25.237

The above is the IP address of the UAT server for scenarios pertaining to Requery API.

UAT server: The UAT server needs to be whitelisted at the merchant's end so that we can post on the merchant side.

9. RESPONSE CODES

Status Code	Description
OTS0000	Refund Initiated Successfully
OTS0625	Transaction ID Not Present
OTS0508	Invalid Amount
OTS0001	Partial Refund Initiated Successfully
OTS0701	Insufficient Amount In The Account To Be Refunded
OTS0671	Duplicate Transaction
OTS0703	Refund Is Not allowed
OTS0623	Unable To Process Refund For HOLD Transaction
OTS0624	Invalid Transaction ID As ChargeBack Raised For The Same Transaction
OTS0625	Duplicate Product Refund Id
OTS0551	Pending From Bank
OTS0600	Invalid Transaction ID Or Failed Transaction
OTS0506	Signature mismatched
OTS0511	Invalid merchId
OTS0512	merchId Mismatched
OTS0522	Invalid Password
OTS0513	Invalid merchTxnId
OTS0514	Invalid txnCurrency
OTS0515	Sum of prodRefundAmount should be equal to totalRefundAmount
OTS0516	This is a single product transaction
OTS0519	Invalid product name
OTS0520	Invalid product amount
OTS0521	TotalRefundAmount is greater than transaction amount

Refund unavailable for Transaction type:

1. NEFT/RTGS or Challan Transactions
Discriminator: NR/CH
2. Wallet Transactions
Discriminator: MW/PW